

گزارش فنی معماری و API های فاز سوم

پروژه سامانه جامع رزرو و خرید بلیط مسابقات ورزشی (BookingSportTickets)

Upstash Redis (Caching & TTL)

PostgreSQL (Neon.tech - Raw SQL)

FastAPI (ASGI)

JWT & RBAC Security

Elasticsearch (Async Indexing)

۱. هدف و خلاصه اجرایی فاز سوم

هدف اصلی از فاز سوم پروژه، پیاده سازی سرویس بک اند مدرن، مقیاس پذیر و با کارایی بالا (High Performance) بر پایه **FastAPI** است. در این فاز، تمامی منطق های کسب و کار از جمله احراز هویت با کد یک بار مصرف (OTP)، مدیریت نشست ها، جستجو و فیلتر پیشرفته بلیط ها، قفل موقت صندلی ها جهت جلوگیری از Overbooking، پردازش پرداخت، مدیریت جریمه کنسلی و پنل پشتیبانی/مدیریت پیاده سازی شده اند.

۲. پشته فناوری (Tech Stack) و علل انتخاب

PostgreSQL & Raw SQL

عدم استفاده از ORM طبق الزامات پروژه جهت بهینه سازی مستقیم کوئری ها، بهره گیری از Stored Procedure های نوشته شده در فاز ۲ و کنترل دقیق اتصال ناهمگام (AsyncPG).

FastAPI & Pydantic

عملکرد Asynchronous فوق العاده بالا بر پایه Starlette، اعتبارسنجی دقیق و خودکار ورودی/خروجی ها با Pydantic Schemas، و تولید خودکار مستندات OpenAPI / Swagger.

Elasticsearch

Upstash Redis

نمایه‌سازی (Indexing) ناهمگام بلیط‌ها برای امکان جستجوی متنی فوق‌سریع و فیلترهای چندبعدی (ورزش، تاریخ، قیمت) بدون اعمال فشار به دیتابیس اصلی.

ذخیره‌سازی کدهای OTP با اعتبار زمان‌دار (TTL)، مدیریت قفل موقت رزروها (۱۰ دقیقه مهلت پرداخت) و کش کردن استعلام‌های پرتکرار (شهرها، ورزشگاه‌ها و پروفایل).

۳. ساختار ماژولار و ماژول‌های هسته (app.core)

ساختار پروژه با رعایت دقیق اصل تفکیک مسئولیت‌ها (Separation of Concerns) به سه لایه اصلی تقسیم شده است:

ماژول‌های هسته (app/core)

- `config.py`: مدیریت مرکزی تنظیمات و متغیرهای محیطی با استفاده از `pydantic-settings`.
- `database.py`: مدیریت پادمان و `Connection Pool` ناهمگام دیتابیس `PostgreSQL`.
- `cache_utils.py` & `redis_client.py`: اتصال به `Redis`، تعریف کلیدواژه‌های کش، و توابع پاک‌سازی کش (`Cache Invalidation`).
- `sync_tickets.py` & `elasticsearch.py`: پی‌کربندی `AsyncElasticsearch` و توابع همگام‌سازی ناهمگام داده‌های بلیط.
- `security.py`: هش کردن کلمات عبور با الگوریتم `Bcrypt`، تولید و اعتبارسنجی توکن‌های `JWT` `Access Token`.
- `dependencies.py`: تزریق وابستگی (`Dependency Injection`) برای استخراج کاربر جاری و اعمال سطح دسترسی نقش‌ها (`RBAC`).

۴. کاتالوگ و تشریح کارکرد APIها (Endpoints)

جدول زیر تمامی مسیرهای پیاده‌سازی‌شده در سرویس بک‌اند، فایل مربوطه، نحوه عمل و تکنولوژی‌های درگیر را نشان می‌دهد:

ماژول / مسیر	متد	عنوان API	نحوه عملکرد و جزئیات پیاده‌سازی
routers/auth.py /auth/otp/send	POST	ارسال کد OTP	تولید کد ۵ رقمی تصادفی و ذخیره در Redis با TTL برابر ۵ دقیقه به همراه شماره موبایل کاربر.
routers/auth.py /auth/otp/verify	POST	تأیید کد OTP	مقایسه کد ارسالی کاربر با مقدار ذخیره‌شده در Redis و تایید صحت احراز هویت.
routers/auth.py /auth/signup	POST	ثبت‌نام کاربر	هش کردن رمز عبور با Bcrypt، درج کاربر در PostgreSQL، و صدور توکن JWT.
routers/auth.py /auth/login	POST	ورود به سیستم	اعتبارسنجی نام کاربری/رمز عبور و صدور Bearer JWT Access Token.
routers/users.py /users/me	GET	دریافت پروفایل	ابتدا از کش Redis خوانده می‌شود؛ در صورت عدم وجود (Cache Miss) از PostgreSQL استعلام و در کش ذخیره می‌گردد.
routers/users.py /users/me	PUT	ویرایش پروفایل	به‌روزرسانی داده‌ها در دیتابیس و پاک‌سازی صریح کش پروفایل کاربر در Redis (Invalidation).
routers/venues.py /venues/cities	GET	لیست شهرها و ورزشگاه‌ها	استعلام لیست شهرهای دارای ورزشگاه همراه با Redis در Cache-Aside جهت کاهش بار دیتابیس.

ماژول / مسیر	متد	عنوان API	نحوه عملکرد و جزئیات پیاده‌سازی
routers/tickets.py /tickets	GET	جستجو و فیلتر بلیط‌ها	اجرای کوئری‌های فیلتر متنی/رنگ/قیمت/ورزش بر روی نمایه Elasticsearch برای پاسخ‌دهی میلی‌ثانیه‌ای.
routers/tickets.py /tickets/{id}	GET	جزئیات بلیط ورزشی	دریافت مشخصات کامل مسابقه، صندلی‌های باقیمانده و ویژگی‌های اختصاصی (فوتبال/والیبال/بسکتبال).
routers/reservations.py /reservations	POST	رزرو موقت بلیط	ایجاد قفل موقت روی صندلی در Redis با TTL برابر ۱۰ دقیقه برای جلوگیری از رزرو همزمان (Overbooking).
routers/reservations.py /reservations/{id}/pay	POST	نهایی‌سازی پرداخت	بررسی قفل موقت، کسر از کیف پول یا درگاه mock، ثبت نهایی خرید در PostgreSQL و همگام‌سازی با Elasticsearch.
routers/reservations.py /reservations/me	GET	تاریخچه خریدهای کاربر	فراخوانی Stored Procedure مربوط به بلیط‌های خریداری شده کاربر به همراه جزئیات وضعیت.
routers/cancellations.py /cancellations/penalty-check	GET	پیش‌نمایش جریمه کنسلی	محاسبه درصد جریمه بر اساس زمان باقیمانده تا

ماژول / مسیر	متد	عنوان API	نحوه عملکرد و جزئیات پیاده‌سازی
			شروع مسابقه (قوانین استاندارد).
<code>routers/cancellations.py</code> <code>/reservations/{id}/cancel</code>	POST	ثبت کنسلی و استرداد	تغییر وضعیت رزرو، عودت مبلغ پس از کسر جریمه به کیف پول کاربر و آزادکننده صندلی در دیتابیس و Elasticsearch.
<code>routers/reports.py</code> <code>/reports</code>	POST	ثبت گزارش مشکل	ثبت اشکالات مربوط به بلیط یا عدم تایید پرداخت توسط کاربر جهت بررسی توسط پشتیبانی.
<code>routers/admin.py</code> <code>/admin/reservations/flagged</code>	GET	شناسایی رزروهای مشکوک	استعلام رزروهایی که رفتارهای مشکوک یا چندباره داشته‌اند (نیاز به دسترسی نقش Admin/Support).

۵. جریان‌های کاری کلیدی (Workflows)

الگوی رزرو موقت و پرداخت بدون Overbooking

1. کاربر درخواست رزرو بلیط را ارسال می‌کند.
2. سرویس بک‌اند با دستور SETNX در Redis صندلی را به مدت ۱۰ دقیقه به نام کاربر قفل می‌کند.
3. در صورت موفقیت‌آمیز بودن پرداخت در مهلت مقرر، خرید در PostgreSQL ثبت شده و وضعیت صندلی قطعی می‌گردد.

4. در صورت انقضای زمان (Expiration) یا انصراف، کلید Redis منقضی شده و صندلی برای سایر کاربران آزاد می‌شود.

⚡ استراتژی مدیریت کش و Invalidation

از الگوی **Cache-Aside** استفاده شده است. هنگام درخواست‌های GET (مانند لیست شهرها یا پروفایل)، ابتدا کش Redis بررسی می‌شود. در صورتی که عملیات تغییر داده (PUT/POST/DELETE) انجام شود، تابع `invalidate_cache()` فراخوانی شده و کلیدهای مربوطه در Redis بلافاصله پاک می‌شوند تا داده‌های قدیمی به کاربر نمایش داده نشوند.

۶. امنیت و کنترل دسترسی (RBAC)

احراز هویت بر پایه **JSON Web Token (JWT)** با الگوریتم HMAC-SHA256 انجام می‌پذیرد. توکن در هدر `Authorization: Bearer <token>` ارسال می‌شود. کنترل سطح دسترسی در فایل `dependencies.py` با متد `check_roles` پیاده‌سازی شده است که نقش‌های کاربر (Customer, Support, Admin) را ارزیابی کرده و در صورت عدم داشتن دسترسی لازم، خطای HTTP 403 Forbidden صادر می‌کند.